

Manual Técnico - SIEA

1. Objetivo

Este documento describe la arquitectura, componentes, configuración, APIs, base de datos y procedimientos de mantenimiento del sistema **SIEA**.

2. Alcance técnico

Incluye:

- Frontend (HTML/CSS/JS)
- Backend PHP (endpoints en api/)
- Base de datos MySQL (siea_db)
- Flujo de autenticación (admin y estudiante)
- Módulos funcionales y operaciones CRUD
- Despliegue local con XAMPP

3. Stack tecnológico

- **Frontend:** HTML5, CSS3, JavaScript (Vanilla)
- **Backend:** PHP 8+ con PDO
- **Base de datos:** MySQL/MariaDB
- **Servidor local:** XAMPP (Apache + MySQL)
- **Librerías cliente (CDN):**
 - qrcode
 - html5-qrcode
 - xlsx
 - html2canvas
 - jspdf

4. Estructura del proyecto

`text

SIAE/

■■ index.html

```
■■ css/
■ ■■ estilos.css
■■ js/
■ ■■ app.js
■■ api/
■ ■■ config.php
■ ■■ utils.php
■ ■■ admin_login.php
■ ■■ student_login.php
■ ■■ student_profile.php
■ ■■ students.php
■ ■■ teachers.php
■ ■■ subjects.php
■ ■■ grades.php
■ ■■ grades_bulk.php
■ ■■ events.php
■ ■■ appointments.php
■ ■■ specialties.php
■ ■■ specialty_materials.php
■■ imagenes/
■ ■■ dgeti.png
■■ siea_database.sql
■■ MANUAL_USUARIO.md
■■ MANUAL_USUARIO.pdf
■■ scripts/
■■ generar_manual_pdf.py
\
```

5. Arquitectura general

5.1. Patrón

Arquitectura cliente-servidor:

- El frontend renderiza vistas y envía solicitudes `fetch`.
- El backend expone endpoints JSON por dominio.

- La capa de persistencia usa PDO con sentencias preparadas.

5.2. Flujo de datos

1. `js/app.js` consume endpoints en `api/` mediante `apiRequest()`.
2. Los endpoints validan método HTTP y payload.
3. Se ejecutan consultas SQL en MySQL.
4. El backend responde en formato JSON estandarizado.
5. El frontend actualiza tablas/módulos y muestra notificaciones.

6. Configuración del backend

6.1. Conexión a BD

Archivo: `api/config.php`

- Host: `127.0.0.1`
- DB: `siea_db`
- User: `root`
- Password: `' '` (vacío por defecto local)
- Charset: `utf8mb4`

Función principal:

- `getPDO(): PDO`

6.2. Utilidades JSON

Archivo: `api/utils.php`

- `jsonResponse(array $payload, int $status = 200): void`
- `readJsonBody(): array`

Estándar de respuesta:

```
`json
```

```
{
  "success": true,
  "data": [...]
}
```

```
`
```

o en error:

```
`json
{
  "success": false,
  "message": "...
}
```

7. Autenticación y sesión

7.1. Login administrador

- Endpoint: POST `api/admin_login.php`
- Flujo:
- Valida usuario fijo (Admin123) y contraseña.
- Verifica/crea registro en tabla administradores con hash bcrypt.
- Inicia sesión PHP (`$_SESSION`).

7.2. Login estudiante

- Endpoint: POST `api/student_login.php`
- Credenciales:
- `numero_control`
- `fecha_nacimiento`
- Consulta tabla `alumnos` con `activo = 1`.

7.3. Observación técnica

Actualmente no hay middleware centralizado de autorización por rol en todos los endpoints; se recomienda como mejora futura.

8. Catálogo de endpoints API

8.1. Auth

- POST `api/admin_login.php` -> login admin
- POST `api/student_login.php` -> login estudiante
- POST `api/student_profile.php` -> actualización perfil/fecha de acceso (según acción)

8.2. Alumnos

- GET api/students.php -> listar
- POST api/students.php -> crear
- PUT api/students.php?id={id} -> actualizar
- DELETE api/students.php?id={id} -> eliminar lógico/físico según implementación

8.3. Docentes

- GET api/teachers.php
- POST api/teachers.php
- PUT api/teachers.php?id={id}
- DELETE api/teachers.php?id={id}

8.4. Materias y calificaciones

- GET api/subjects.php -> materias activas
- GET api/grades.php -> lista de calificaciones
- POST api/grades.php -> registrar calificación
- POST api/grades_bulk.php -> carga masiva desde Excel procesado en frontend

8.5. Eventos

- GET api/events.php
- POST api/events.php
- PUT api/events.php?id={id}
- DELETE api/events.php?id={id}

8.6. Citas

- GET api/appointments.php
- POST api/appointments.php
- PUT api/appointments.php?id={id}
- DELETE api/appointments.php?id={id}

8.7. Especialidades y materiales

- GET api/specialties.php
- POST api/specialties.php
- PUT api/specialties.php?id={id}

- DELETE `api/specialties.php?id={id}`
- POST `api/specialty_materials.php` -> alta/actualización de cuadernillos y formatos

9. Frontend técnico (js/app.js)

9.1. Elementos clave

- Estado global: `currentUser`, `currentStudent`, `demoData`
- Cliente API: `apiRequest(endpoint, options)`
- Navegación:
- `showPage()`
- `showModule()`
- `showModuleStudent()`

9.2. Operaciones principales

- Carga de datos:
- `loadStudents()`, `loadTeachers()`, `loadGrades()`, `loadEvents()`, `loadAppointments()`, `loadSpecialties()`
- Formularios CRUD:
- `handleStudentSubmit()`, `handleTeacherSubmit()`, `handleGradeSubmit()`, etc.
- Utilidades UI:
- `showToast()`
- skeleton loading de tablas
- estados de botón `is-loading`
- contadores animados para KPIs

9.3. Integración API

`API_BASE = 'api'`

Ejemplo de solicitud:

```
`js
await apiRequest('students.php', {
  method: 'POST',
  body: JSON.stringify(payload)
});
`
```

10. Estilos y UI (css/estilos.css)

- Variables CSS en :root (colores, spacing, sombras, radios).
- Layout principal:
- login
- dashboard admin/estudiante
- sidebar + módulos
- tablas y modales
- Mejoras recientes:
- microinteracciones
- focus-visible
- toasts con barra de progreso
- skeleton shimmer
- soporte prefers-reduced-motion

11. Base de datos

Archivo de referencia: siea_database.sql

Entidades principales:

- administradores
- alumnos
- docentes
- materias
- calificaciones
- eventos
- citas
- especialidades
- materiales asociados (cuadernillos/formatos, según modelo en API)

Relaciones funcionales destacadas:

- alumnos.especialidad_id -> especialidades.id
- calificaciones.alumno_id -> alumnos.id
- calificaciones.materia_id -> materias.id

- `calificaciones.docente_id -> docentes.id`
- `citas.alumno_id -> alumnos.id`
- `citas.docente_id -> docentes.id`

12. Despliegue local (entorno de desarrollo)

1. Clonar repositorio en htdocs:

- `git clone https://github.com/fblancor-del/SIAE.git`

2. Iniciar XAMPP:

- Apache
- MySQL

3. Importar BD:

- `siea_database.sql` en phpMyAdmin

4. Abrir app:

- `http://localhost/SIAE/`

13. Operación de mantenimiento

13.1. Backup

- Exportar `siea_db` desde phpMyAdmin (formato SQL).

13.2. Restauración

- Crear BD y ejecutar importación SQL.

13.3. Versionado

- Flujo recomendado:
- `git pull`
- cambios
- `git add -A`
- `git commit -m "..."`
- `git push`

14. Seguridad técnica (estado actual y mejoras)

14.1. Ya implementado

- PDO + prepared statements
- Hash de contraseña admin en BD (password_hash / password_verify)
- Validación de métodos HTTP
- Respuestas JSON estables

14.2. Recomendado (siguiente fase)

- Middleware de sesión/rol en todos los endpoints
- Rate limiting para login
- Rotación de credenciales por entorno
- CSRF token para operaciones críticas
- Logs de auditoría por acción administrativa

15. Troubleshooting técnico

Error: Respuesta invalida del servidor

- Verificar que el endpoint PHP no emita HTML/warnings.
- Revisar errores en consola de navegador y logs Apache/PHP.

Error: No se pudieron cargar ... de la base de datos

- Confirmar MySQL activo.
- Revisar credenciales en api/config.php.
- Verificar que siea_db exista y tenga tablas.

Login admin no funciona

- Confirmar credenciales de entrada.
- Verificar tabla administradores y campo activo.

Recursos estáticos no cargan

- Verificar rutas css/estilos.css, js/app.js, imagenes/dgeti.png.
- Limpiar caché con Ctrl + F5.

16. Convenciones técnicas del proyecto

- Respuesta API en JSON con bandera success.
- Manejo de errores con try/catch en frontend y backend.
- Nombres de funciones por dominio funcional.
- Cambios UI desacoplados de lógica de negocio.

17. Hoja de ruta técnica sugerida

1. Extraer módulos JS por dominio (auth, students, events, etc.).
2. Implementar control de acceso backend por sesión/rol.
3. Incorporar validaciones centralizadas en API.
4. Añadir pruebas básicas de endpoints críticos.
5. Documentar contrato OpenAPI (Swagger) para integración futura.

Versión: 1.0

Fecha: 25/02/2026

Sistema: SIEA